



UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Departamento de Ciencias de la Computación

Construcción de Compiladores

Fase de Compilación: Análisis Semántico

Analizador sintáctico y semántico de Compiscript

https://github.com/G1LB3T0/Proyecto1_Compis_Generador_Analizador_Lexico/tree/compiladores-2

Catedrático

Ing. Pablo Koch

Integrantes

Luis Gonzalez — 23353

Joel Jaquez — 23369

Guatemala, 6 de septiembre de 2026

Índice

1. Introducción	3
2. Objetivos	3
2.1. Objetivo general	3
2.2. Objetivos específicos	3
3. Arquitectura general	3
3.1. El pipeline	3
3.2. Núcleo independiente de la interfaz	4
4. Estructura del repositorio y trazabilidad por commits	4
4.1. Historial de la rama	5
4.2. Lectura del historial	6
5. Descripción de los archivos por capa	6
5.1. Capa de gramática y código generado	6
5.2. Núcleo del analizador	7
5.3. Pruebas, IDE y construcción	7
6. Gramática y generación con ANTLR	7
7. Reglas semánticas implementadas	8
7.1. Ejemplo de programa válido	9
7.2. Ejemplo de programa con errores	9
8. Tabla de símbolos y manejo de ámbitos	10
8.1. Estructura	10
8.2. Declaración en dos momentos	10
8.3. Closures	10
9. El IDE	11
9.1. Diagnósticos	13
9.2. Árbol sintáctico	14
9.3. Tabla de símbolos y ámbitos	14
9.4. Sistema de tipos, tokens y clases	15
10. Batería de pruebas	16
11. Compilación y ejecución	17

11.1. Requisitos	17
11.2. Construcción	17
11.3. Ejecución del IDE	18
11.4. Uso directo del CLI	18
12.Decisiones sobre el material oficial	18
13.Correspondencia con la rúbrica	19
14.Conclusiones	19
15.Referencias	19

1. Introducción

Este informe documenta la implementación del analizador sintáctico y semántico de **Compiscript**, un lenguaje basado en un subconjunto de TypeScript. El trabajo corresponde a la fase de análisis semántico en la construcción de un compilador: partiendo de la gramática oficial en ANTLR se construye el árbol sintáctico, se recorre mediante un *Visitor* y se validan las reglas de tipos, ámbitos, funciones, control de flujo, clases y estructuras de datos, sosteniendo en todo momento una tabla de símbolos con entornos anidados.

La característica que distingue esta implementación es que **todo el compilador está escrito en C++17**. ANTLR se utilizó con su *target* de C++, de modo que el lexer, el parser y el visitor generados se compilan y enlazan junto con el analizador semántico en una sola biblioteca nativa. Python aparece únicamente como adaptador web: un servidor Flask de 139 líneas que sirve el IDE y traduce peticiones HTTP en invocaciones al ejecutable C++. Ninguna regla léxica, sintáctica o semántica está implementada en Python ni en JavaScript.

El sistema acepta archivos con extensión `.cps` y produce, en un único documento JSON, la lista de tokens, el árbol sintáctico serializado, los diagnósticos con ubicación exacta y el estado completo de la tabla de símbolos por cada entorno creado.

2. Objetivos

2.1. Objetivo general

Implementar un analizador semántico funcional para Compiscript, aplicando los conceptos de sistemas de tipos, ámbitos y entornos de ejecución sobre un árbol sintáctico real, con una tabla de símbolos capaz de sostener las fases posteriores del compilador.

2.2. Objetivos específicos

- Implementar el analizador sintáctico de Compiscript utilizando ANTLR.
- Construir el árbol sintáctico con representación visual navegable.
- Recorrer el árbol mediante un *Visitor* de ANTLR para evaluar las reglas semánticas del lenguaje.
- Implementar una batería de pruebas que valide casos exitosos y fallidos para cada regla semántica.
- Desarrollar un IDE que permita escribir, cargar y compilar código Compiscript.

3. Arquitectura general

3.1. El pipeline

El flujo de información va del texto fuente al documento JSON que consume el IDE. Cada etapa tiene una responsabilidad única y un artefacto de salida observable:

```

Código .cps
|
v
CompiscriptLexer (ANTLR, C++)
| tokens
v
CompiscriptParser (ANTLR, C++)
| parse tree
|-----> tree_serializer -> árbol JSON -> IDE
|
v
SemanticAnalyzer (Visitor C++, deriva de CompiscriptBaseVisitor)
|
|-- sistema de tipos y compatibilidad
|-- resolución léxica de nombres (tabla de símbolos)
|-- funciones, recursión y closures
|-- control de flujo y código muerto
|-- clases, herencia, miembros y constructores
|-- listas e índices
|
v
JSON: diagnósticos + scopes + símbolos + clases
|
v
Flask (adaptador) -> IDE web

```

3.2. Núcleo independiente de la interfaz

El núcleo C++ expone una sola función pública, `analyzeSource`, que recibe el texto del programa y devuelve un `AnalysisResult`. Los tres consumidores del sistema —el CLI, la batería de pruebas y el servidor Flask— utilizan esa misma API. Esta decisión evita el problema clásico de tener implementaciones divergentes del compilador para la línea de comandos y para la interfaz gráfica: lo que prueba la batería es exactamente lo que ejecuta el IDE.

Otra decisión relevante es que **el análisis semántico no se ejecuta cuando existen errores léxicos o sintácticos**. Recorrer un árbol recuperado e incompleto produce diagnósticos en cascada que confunden más de lo que ayudan. Aun así, los tokens y el árbol se conservan y se envían al IDE, porque son justamente la información que el usuario necesita para corregir la entrada.

4. Estructura del repositorio y trazabilidad por commits

El desarrollo de esta fase se realizó en la rama `compiladores-2`. Cada commit corresponde a una unidad funcional completa y a un solo autor, de modo que la contribución individual queda registrada en el historial. En Git se utilizaron tres identidades que corresponden a los dos integrantes:

Identidad en Git	Correo	Integrante
G1LB3T0	luis.yo09@hotmail.com	Luis Gonzalez (23353)
jaq23369	jaq23369@uvg.edu.gt	Joel Jaquez (23369)
Joel Jaquez	123708276+jaq23369@...	Joel Jaquez (23369)

Tabla 1: Identidades de Git utilizadas en la rama.

4.1. Historial de la rama

La tabla siguiente recorre los ocho commits de la rama en orden cronológico y explica qué archivos introduce cada uno y por qué. Es, en la práctica, el mapa de lectura del repositorio.

Commit	Autor	Aporte y archivos
6de9fe5	Joel Jaquez	[INIT] Preparar la base del proyecto Compiscript. Incorpora el material oficial en <code>Instrucciones/</code> : el enunciado, la especificación del lenguaje y la gramática entregada <code>Compiscript (1).g4</code> . Ajusta <code>.gitignore</code> y el <code>README.md</code> para separar el trabajo de esta fase del de Compiladores 1.
ac9e6cf	Luis Gonzalez	[ADD] Configurar ANTLR y compilación de Compiscript. Es el commit que levanta la infraestructura. Añade <code>backend/compiscript/grammar/Compiscript.g4</code> (la gramática activa, ya extendida), los doce archivos de <code>backend/compiscript/generated/</code> producidos por ANTLR con el <code>target</code> de C++, el <code>CMakeLists.txt</code> que compila el runtime C++ de ANTLR junto con el proyecto, y los scripts reproducibles <code>scripts/build_compiscript.sh</code> , <code>.ps1</code> y <code>generate_antlr.sh</code> .
dafd076	Joel Jaquez	[ADD] Implementar análisis semántico de Compiscript. El corazón del proyecto. Introduce <code>include/compiscript/model.h</code> (símbolos, scopes y clases), <code>include/compiscript/semantic_analyzer.h</code> y las 1 218 líneas de <code>src/semantic_analyzer.cpp</code> , que es el Visitor propiamente dicho, más <code>src/symbol_table.cpp</code> con la mecánica de declaración y resolución de nombres.
f911f64	Luis Gonzalez	[ADD] Integrar servicio C++ y salida del analizador. Convierte el Visitor en un servicio consumible. Añade <code>service.{h,cpp}</code> (orquestración del pipeline y <code>analyzeSource</code>), <code>diagnostics.{h,cpp}</code> (código estable, categoría, severidad, línea y columna), <code>tree_serializer.{h,cpp}</code> (parse tree → JSON), <code>token_classifier.{h,cpp}</code> (categoría legible de cada token), <code>json.{h,cpp}</code> (serialización sin dependencias externas) y <code>main.cpp</code> , el ejecutable <code>compiscript_cli</code> .
a1ffefe	Joel Jaquez	[ADD] Desarrollar IDE de análisis para Compiscript. Construye la interfaz: <code>frontend/templates/index.html</code> , <code>frontend/static/js/app.js</code> (1 001 líneas: editor, árbol interactivo, tablas y navegación de diagnósticos), <code>frontend/static/css/style.css</code> , el adaptador <code>app.py</code> y <code>requirements.txt</code> .

Commit	Autor	Aporte y archivos
49af6b1	Luis Gonzalez	[TEST] Cubrir reglas semánticas de Compiscript. Añade <code>backend/tests/compiscript_tests.cpp</code> (654 líneas, 95 escenarios) y los tres programas de demostración <code>backend/examples/compiscript/: bienvenido.cps</code> , <code>closures.cps</code> y <code>errores_semanticos.cps</code> .
a40593a	Joel Jaquez	[DOC] Documentar arquitectura y evaluación de Compiscript. Crea <code>docs/arquitectura-compiscript.md</code> y <code>docs/decisiones-compiscript.md</code> , reescribe el <code>README.md</code> y elimina archivos de planificación que ya no correspondían al estado del proyecto.
12bfe89	Joel Jaquez	[ADD] Comentarios importantes. Pasada final de documentación en el código: comenta los 25 archivos fuente del núcleo, las pruebas, el frontend y los scripts, explicando la intención de cada módulo.

Tabla 2: Trazabilidad de la rama `compiladores-2`.

4.2. Lectura del historial

El orden de los commits refleja el orden natural de construcción de un compilador y conviene leerlo así: primero la gramática y la generación (`ac9e6cf`), luego el Visitor que da significado al árbol (`dafd076`), después la capa que expone los resultados (`f911f64`), sobre ella la interfaz (`a1ffefe`), y finalmente la evidencia de que todo lo anterior funciona (`49af6b1`) y la explicación de por qué está hecho así (`a40593a`, `12bfe89`).

5. Descripción de los archivos por capa

5.1. Capa de gramática y código generado

Archivo	Líneas	Función
<code>grammar/Compiscript.g4</code>	115	Fuente de verdad sintáctica. Define tokens y reglas del lenguaje.
<code>generated/CompiscriptLexer.*</code>	—	Autómata léxico generado por ANTLR.
<code>generated/CompiscriptParser.*</code>	—	Parser ALL(*) y contextos de regla.
<code>generated/CompiscriptVisitor.*</code>	—	Interfaz del patrón Visitor.
<code>generated/CompiscriptBaseVisitor.*</code>	—	Implementación por defecto de la que hereda el analizador.

Tabla 3: Archivos de la capa sintáctica.

El código generado se versiona en el repositorio a propósito: permite compilar el proyecto sin Java disponible, mientras que los scripts permiten regenerarlo cuando la gramática cambia.

5.2. Núcleo del analizador

Archivo	Líneas	Función
src/semantic_analyzer.cpp	1218	El Visitor. Sintetiza tipos, aplica reglas contextuales y emite diagnósticos.
include/.../model.h	127	Estructuras <code>Symbol</code> , <code>Scope</code> , <code>ClassInfo</code> y <code>AnalysisResult</code> .
include/.../semantic_analyzer.h	171	Declaración del Visitor y su estado contextual.
src/json.cpp	225	Serializador JSON propio, sin dependencias externas.
src/service.cpp	76	Orquesta lexer, parser, serializador y Visitor.
src/tree_serializer.cpp	68	Convierte el parse tree de ANTLR en JSON recursivo.
src/token_classifier.cpp	65	Asigna categoría legible a cada token.
src/main.cpp	53	Ejecutable <code>compiscript_cli</code> .
src/symbol_table.cpp	38	Declaración y resolución de nombres en la cadena de ámbitos.
src/diagnostics.cpp	21	Construcción uniforme de diagnósticos.

Tabla 4: Archivos del núcleo C++.

5.3. Pruebas, IDE y construcción

Archivo	Líneas	Función
tests/compiscript_tests.cpp	654	95 escenarios enlazados contra <code>compiscript_core</code> .
frontend/static/js/app.js	1001	Editor, árbol interactivo, tablas y navegación de errores.
frontend/static/css/style.css	235	Hoja de estilos del IDE.
frontend/templates/index.html	101	Estructura de la interfaz.
app.py	139	Adaptador Flask. Cuatro endpoints.
CMakeLists.txt	—	Compila el runtime de ANTLR, la biblioteca, el CLI y las pruebas.
scripts/build_compiscript.*	—	Descarga ANTLR, regenera, compila y ejecuta la batería.

Tabla 5: Archivos de pruebas, interfaz y construcción.

El servidor expone cuatro endpoints: `/api/files` y `/api/file/<nombre>` para listar y leer ejemplos, `/api/save` para guardar, `/api/analyze` que ejecuta `compiscript_cli` con la fuente por entrada estándar, y `/api/tests` que ejecuta el binario de la batería. En los dos últimos casos Flask se limita a reenviar el JSON producido por C++.

6. Gramática y generación con ANTLR

La gramática activa reside en `backend/compiscript/grammar/Compiscript.g4` y se genera con el *target* de C++ y el patrón Visitor:

```
1 java -jar tools/antlr-4.13.2-complete.jar -Dlanguage=C++ -visitor \
2     -no-listener -o backend/compiscript/generated -Xexact-output-dir \
```


Listing 1: Generación del lexer, parser y Visitor.

Se eligió *Visitor* y no *Listener* porque el análisis semántico necesita **sintetizar** valores: el tipo de una expresión se obtiene a partir del tipo de sus subexpresiones, y el Visitor permite que cada `visitX` devuelva ese tipo hacia el nodo padre. Con un Listener habría que mantener manualmente una pila de resultados.

7. Reglas semánticas implementadas

El Visitor emite 32 códigos de diagnóstico distintos, todos con categoría, severidad, línea y columna. La tabla siguiente los agrupa por la familia de reglas que exige el enunciado.

Familia	Comportamiento implementado y códigos
Sistema de tipos	Verifica <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> y <code>%</code> sobre números; <code>&&</code> , <code> </code> y <code>!</code> sobre booleanos; comparaciones compatibles; tipos de asignación; e inicialización obligatoria de <code>const</code> . Reconoce <code>integer</code> , <code>float</code> , <code>string</code> , <code>boolean</code> , <code>null</code> , <code>void</code> , clases y listas, con ampliación segura <code>integer</code> → <code>float</code> . SEM_ARITHMETIC_TYPE, SEM_LOGICAL_TYPE, SEM_COMPARE_TYPE, SEM_ASSIGN_TYPE, SEM_ASSIGN_CONST, SEM_TERNARY_TYPE, SEM_TYPE_UNKNOWN, SEM_TYPE_VOID.
Manejo de ámbito	Resuelve primero el entorno local y continúa por los padres hasta el global; reporta nombres no declarados y redeclaraciones; permite sombreado en entornos hijos; crea un scope por programa, función, clase y bloque. SEM_SCOPE_UNDECLARED, SEM_SCOPE_DUPLICATE.
Funciones y procedimientos	Conserva firmas completas, valida cantidad y tipo posicional de argumentos, comprueba el tipo de retorno y los caminos sin retorno, admite recursión y funciones anidadas, y registra las capturas de closures. SEM_FUNCTION_ARITY, SEM_FUNCTION_ARGUMENT, SEM_FUNCTION_RETURN, SEM_FUNCTION_MISSING_RETURN, SEM_FUNCTION_NOT_CALLABLE.
Control de flujo	Exige condición booleana en <code>if</code> , <code>while</code> , <code>do-while</code> , <code>for</code> y <code>switch</code> ; limita <code>break</code> y <code>continue</code> a bucles y <code>return</code> a funciones; detecta instrucciones inalcanzables. SEM_FLOW_BREAK, SEM_FLOW_CONTINUE, SEM_FLOW_RETURN, SEM_FLOW_DEAD_CODE, SEM_SWITCH_CASE.
Clases y objetos	Comprueba miembros accedidos con <code>.</code> , firma del constructor al ejecutar <code>new</code> , uso contextual de <code>this</code> , herencia simple, existencia de la clase base, ausencia de ciclos y compatibilidad de subtipo. SEM_CLASS_MEMBER, SEM_CLASS_PROPERTY, SEM_CLASS_CONSTRUCTOR, SEM_CLASS_THIS, SEM_CLASS_BASE_UNKNOWN, SEM_CLASS_CYCLE, SEM_CLASS_UNKNOWN.

Familia	Comportamiento implementado y códigos
Listas y estructuras	Infiere un tipo común para los elementos, admite listas anidadas y listas vacías tipadas, exige índices <code>integer</code> y valida que <code>foreach</code> reciba una lista. SEM_LIST_ELEMENT, SEM_LIST_INDEX, SEM_LIST_ACCESS, SEM_LIST_FOREACH.
Reglas generales	Rechaza objetivos de asignación inválidos y operaciones sin sentido, como multiplicar una función, además de las declaraciones duplicadas de variables, funciones y parámetros. SEM_ASSIGN_TARGET, SEM_FUNCTION_NOT_CALLABLE, SEM_SCOPE_DUPLICATE.

Tabla 6: Reglas semánticas exigidas y su implementación.

7.1. Ejemplo de programa válido

```

1  class Animal {
2      let nombre: string;
3
4      function constructor(nombre: string) {
5          this.nombre = nombre;
6      }
7
8      function hablar(): string {
9          return this.nombre + " hace ruido.";
10     }
11 }
12
13 class Perro : Animal {
14     function hablar(): string {
15         return this.nombre + " ladra.";
16     }
17 }
18
19 function factorial(n: integer): integer {
20     if (n <= 1) return 1;
21     return n * factorial(n - 1);
22 }
23
24 let perro: Perro = new Perro("Toby");
25 let notas: integer[] = [90, 85, 100];
26
27 foreach (nota in notas) {
28     print(nota);
29 }

```

Listing 2: bienvenido.cps: clases, herencia, recursión y listas.

7.2. Ejemplo de programa con errores

El siguiente archivo concentra a propósito una violación de cada familia de reglas. El analizador produce **17 diagnósticos** sobre él.

```

1  const limite: integer = 10;
2  limite = 20;
3

```

```
4 let activo: boolean = 1 + true;
5 print(noDeclarada);
6
7 function sumar(a: integer, a: integer): integer {
8     return "resultado incorrecto";
9     print("codigo muerto");
10 }
11
12 break;
13
14 class Caja {
15     let valor: integer;
16 }
17
18 let caja: Caja = new Caja(1);
19 print(caja.inexistente);
20
21 let datos: integer[] = [1, true, 3];
22 print(datos[false]);
```

Listing 3: errores_semanticos.cps.

8. Tabla de símbolos y manejo de ámbitos

8.1. Estructura

Cada *Scope* almacena un identificador estable, la clase de entorno (*global*, *class*, *function* o *block*), la referencia a su padre, el símbolo propietario cuando se trata de una clase o una función, el mapa local de nombre a símbolo y la lista de sus hijos.

La resolución de un nombre comienza en el ámbito actual y sube por la cadena de padres hasta el global. De ahí se derivan dos comportamientos: una declaración interna *oculta* a una externa del mismo nombre, mientras que una redeclaración dentro del mismo mapa se reporta como error. El modelo corresponde a la tabla por ámbito y a la regla del entorno más cercano descritas en la sección 2.7 del libro del dragón.

Una decisión deliberada es que **los scopes no se destruyen al salir de un bloque**: el Visitor únicamente restaura el identificador del padre. Gracias a esto el resultado final conserva una instantánea navegable de todos los entornos creados durante el análisis, que es exactamente lo que la evaluación pide mostrar.

8.2. Declaración en dos momentos

Las clases y funciones se registran en una pasada de recolección previa al análisis de sus cuerpos. Esto habilita cuatro comportamientos necesarios: llamadas recursivas, llamadas a funciones declaradas más adelante en el mismo ámbito, referencias a clases declaradas después, y resolución de miembros heredados sin depender del orden textual. Las variables ordinarias, en cambio, se registran al visitar su declaración, por lo que no pueden usarse antes de declararse.

8.3. Closures

Cuando una función anidada resuelve una variable o parámetro declarado fuera de su propio scope de función, se registran dos relaciones simétricas: ***captures*** en la función y ***captured_by*** en el símbolo capturado. Esto representa estáticamente el entorno de definición que una fase

posterior tendría que convertir en un enlace de acceso o una closure en tiempo de ejecución, como se discute en la sección 7.3 del libro del dragón.

```

1  function crearAcumulador(base: integer): integer {
2      function sumar(valor: integer): integer {
3          return base + valor;
4      }
5
6      return sumar(10);
7  }
8
9  let resultado: integer = crearAcumulador(5);
10 print(resultado);

```

Listing 4: `closures.cps`: la función interna captura `base`.

Diagnósticos 0 Árbol sintáctico Tabla de símbolos Sistema de tipos Manejo de ámbitos Tokens Clases

global

global

scope #0 · padre -1

NOMBRE	CLASE	TIPO / FIRMA	ESTADO	CAPTURAS	UBICACIÓN
crearAcumulador	Función	(integer) → integer	solo lectura	—	1:1
resultado	Variable	integer	mutable · inicializado	—	9:1

function

función crearAcumulador

scope #1 · padre 0

NOMBRE	CLASE	TIPO / FIRMA	ESTADO	CAPTURAS	UBICACIÓN
base	Parámetro	integer	mutable · inicializado	—	1:26

block

bloque@1

scope #2 · padre 1

NOMBRE	CLASE	TIPO / FIRMA	ESTADO	CAPTURAS	UBICACIÓN
sumar	Función	(integer) → integer	solo lectura	base	2:3

function

función sumar

scope #3 · padre 2

NOMBRE	CLASE	TIPO / FIRMA	ESTADO	CAPTURAS	UBICACIÓN
valor	Parámetro	integer	mutable · inicializado	—	2:18

block

bloque@2

scope #4 · padre 3

Figura 1: Tabla de símbolos de `closures.cps`. La función anidada `sumar` registra la captura del parámetro `base`.

9. El IDE

El IDE es una aplicación web servida por Flask. Permite escribir, cargar, guardar y descargar archivos `.cps`; analizar el programa con `Ctrl/Cmd + Enter` o con el botón principal; navegar los diagnósticos saltando a su ubicación; explorar el árbol sintáctico; consultar la tabla de símbolos; revisar el sistema de tipos, los ámbitos, los tokens y las clases; y ejecutar la batería de pruebas desde la propia interfaz.

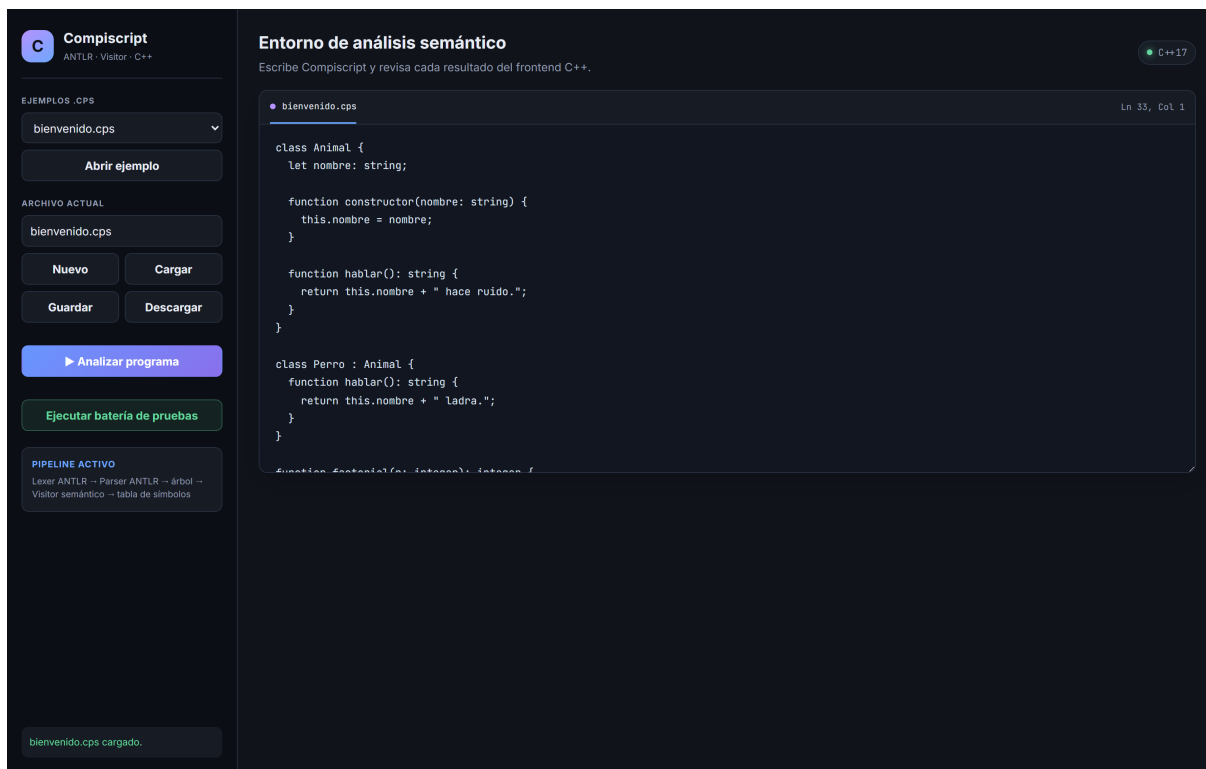


Figura 2: Estado inicial del IDE.

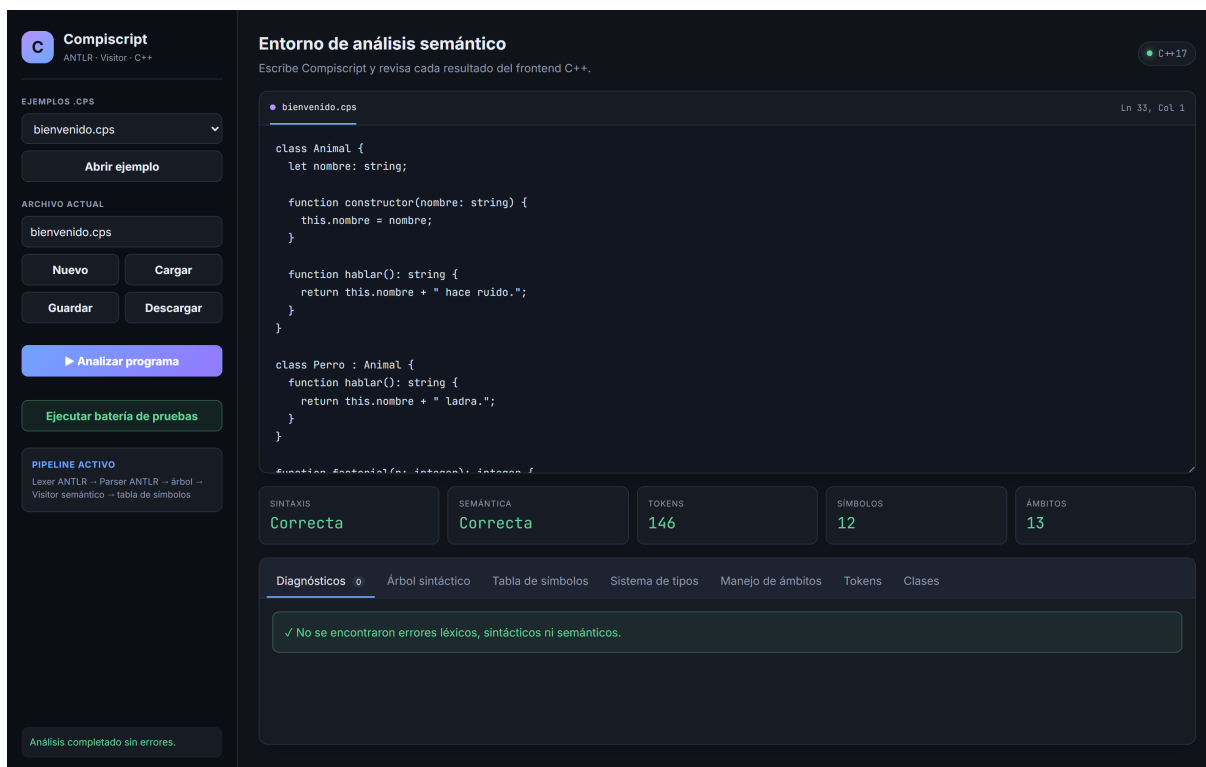


Figura 3: Análisis de bienvenido.cps: sintaxis y semántica correctas, 146 tokens, 12 símbolos y 13 ámbitos.

9.1. Diagnósticos

Cada diagnóstico muestra su código estable, la categoría a la que pertenece, el mensaje y la ubicación línea:columna. Al hacer clic sobre uno, el editor mueve el cursor a esa posición.

The screenshot shows the Compiscript IDE interface. On the left sidebar, there's a section for 'EJEMPLOS .CPS' with a dropdown menu showing 'errores_semanticos.cps' and buttons for 'Abrir ejemplo', 'Nuevo', 'Cargar', 'Guardar', 'Descargar', and 'Ejecutar batería de pruebas'. Below this is the 'PIPELINE ACTIVO' section showing the workflow: 'Lexer ANTLR → Parser ANTLR → Árbol → Visitor semántico → Tabla de símbolos'. The main panel has a code editor at the top with the following code:

```
break;

class Caja {
    let valor: integer;
}

let caja: Caja = new Caja(1);
metodo (sumar):
```

Below the code editor, there are five status boxes: 'SINTAXIS' (Correcta), 'SEMANTICA' (Con errores), 'TOKENS' (100), 'SIMBOLOS' (8), and 'AMBITOS' (4). The 'Diagnósticos' tab is active, showing a list of 17 errors. The first seven errors are highlighted in the image:

- SEM_ASSIGN_CONST Tipo** (2:1): No se puede reasignar la constante 'limite'.
- SEM_ARITHMETIC_TYPE Tipo** (4:23): El operador '+' requiere números (o dos string para +), no 'integer' y 'boolean'.
- SEM_SCOPE_UNDECLARED Ámbito** (5:7): El identificador 'noDeclarada' no está declarado en un ámbito accesible.
- SEM_SCOPE_DUPLICATE Ámbito** (7:28): El identificador 'a' ya fue declarado en este ámbito (línea 7).
- SEM_FUNCTION_RETURN Función** (8:3): La función 'sumar' retorna 'string', pero declaró 'integer'.
- SEM_FLOW_DEAD_CODE Flujo** (9:3): Código inalcanzable después de una instrucción que termina el flujo.
- SEM_FUNCTION_MISSING_RETURN Función** (7:1): La función 'sumar' debe retornar un valor de tipo 'integer' en todos sus caminos.

Other errors in the list include SEM_FLOW_BREAK (12:1) and SEM_FLOW_DEAD_CODE (14:1).

Figura 4: Análisis de `errores_semanticos.cps`: la sintaxis es correcta y la semántica reporta 17 errores.

This is a close-up view of the diagnostics panel from Figure 4. It shows a list of seven errors, each with a category, a message, and a location (line:column):

- SEM_ASSIGN_CONST Tipo** (2:1): No se puede reasignar la constante 'limite'.
- SEM_ARITHMETIC_TYPE Tipo** (4:23): El operador '+' requiere números (o dos string para +), no 'integer' y 'boolean'.
- SEM_SCOPE_UNDECLARED Ámbito** (5:7): El identificador 'noDeclarada' no está declarado en un ámbito accesible.
- SEM_SCOPE_DUPLICATE Ámbito** (7:28): El identificador 'a' ya fue declarado en este ámbito (línea 7).
- SEM_FUNCTION_RETURN Función** (8:3): La función 'sumar' retorna 'string', pero declaró 'integer'.
- SEM_FLOW_DEAD_CODE Flujo** (9:3): Código inalcanzable después de una instrucción que termina el flujo.
- SEM_FUNCTION_MISSING_RETURN Función** (7:1): La función 'sumar' debe retornar un valor de tipo 'integer' en todos sus caminos.

Figura 5: Panel de diagnósticos con las siete familias de reglas representadas.

9.2. Árbol sintáctico

El árbol es la serialización del parse tree que produce ANTLR. Cada nodo indica la regla o el token, su posición y el número de hijos; se puede expandir, contraer y ampliar.



Figura 6: Representación visual del árbol sintáctico.

9.3. Tabla de símbolos y ámbitos

Diagnósticos 0

Árbol sintáctico

Tabla de símbolos

Sistema de tipos

Manejo de ámbitos

Tokens

Clases

global

global

scope #0 · padre -1

NOMBRE	CLASE	TIPO / FIRMA	ESTADO	CAPTURAS	UBICACIÓN
Animal	Clase	Animal	solo lectura	—	1:1
Perro	Clase	Perro	solo lectura	—	13:1
factorial	Función	(integer) → integer	solo lectura	—	19:1
notas	Variable	integer[]	mutable · inicializado	—	25:1
perro	Variable	Perro	mutable · inicializado	—	24:1

class

class Animal

scope #1 · padre 0

NOMBRE	CLASE	TIPO / FIRMA	ESTADO	CAPTURAS	UBICACIÓN
constructor	Método	(string) → void	solo lectura	—	4:3
hablar	Método	() → string	solo lectura	—	8:3
nombre	Atributo	string	mutable · inicializado	—	2:3

function

función constructor

scope #3 · padre 1

NOMBRE	CLASE	TIPO / FIRMA	ESTADO	CAPTURAS	UBICACIÓN
nombre	Parámetro	string	mutable · inicializado	—	4:24

block

bloque@4

scope #4 · padre 3

Figura 7: Tabla de símbolos jerárquica: cada entorno lista sus símbolos con clase, tipo o firma, estado, capturas y ubicación.

Diagnósticos ① Árbol sintáctico Tabla de símbolos Sistema de tipos **Manejo de ámbitos** Tokens Clases

¿Qué demuestra esta tabla?
Cada fila es un entorno creado realmente por el Visitor C++. Para resolver un nombre se revisa primero el entorno actual y, si no aparece, se continúa con su padre hasta llegar al ámbito global.

ID	ENTORNO	TIPO	PADRE	PROPIETARIO	SÍMBOLOS DECLARADOS	LÍNEA
#0	global	Global	— (raíz)	—	Animal, Perro, factorial, notas, perro	1
#1	clase Animal	Clase	#0	Animal	constructor, hablar, nombre	1
#2	clase Perro	Clase	#0	Perro	hablar	13
#3	función constructor	Función	#1	constructor	nombre	4
#4	bloque@4	Bloque	#3	—	—	4
#5	función hablar	Función	#1	hablar	—	8
#6	bloque@8	Bloque	#5	—	—	8
#7	función hablar	Función	#2	hablar	—	14
#8	bloque@14	Bloque	#7	—	—	14
#9	función factorial	Función	#0	factorial	n	19
#10	bloque@19	Bloque	#9	—	—	19
#11	foreach@27	Bloque	#0	—	nota	27
#12	bloque@27	Bloque	#11	—	—	27

Global Función

Figura 8: Vista tabular de ámbitos: identificador, clase de entorno, padre, propietario y símbolos declarados.

9.4. Sistema de tipos, tokens y clases

Diagnósticos ① Árbol sintáctico Tabla de símbolos **Sistema de tipos** Manejo de ámbitos Tokens Clases

¿Cuándo se construye esta información?
Se actualiza cada vez que analizas un archivo .cps. El Visitor C++ determina los tipos de declaraciones, parámetros, retornos, listas y objetos mientras recorre el árbol sintáctico.

Catálogo de tipos activo
Incluye los tipos base del lenguaje y los tipos adicionales encontrados en el programa analizado.

TIPO	CATEGORÍA	SIGNIFICADO SENCILLO	COMPATIBILIDAD PRINCIPAL	USOS EN ESTE .CPS
integer	Primitivo numérico	Números enteros; admite aritmética y comparaciones.	Puede usarse donde se espera integer o promoverse a float.	4
float	Primitivo numérico	Números con decimales; admite aritmética y comparaciones.	Acepta valores float y también integer por promoción.	0
string	Primitivo textual	Cadenas de texto; permite concatenación con +.	Solo es compatible con string; también puede recibir null.	5
boolean	Primitivo lógico	Valores true o false usados por condiciones y operadores lógicos.	Solo es compatible con boolean.	0
null	Valor especial	Representa ausencia de un valor.	Puede asignarse a string, listas y objetos.	0
void	Ausencia de retorno	Indica que una función no devuelve un valor.	No puede utilizarse como tipo de una variable.	1
Animal	Clase del programa	Objeto perteneciente a la clase Animal.	Es compatible con su propia clase y con clases base según la herencia.	1
Perro	Clase del programa	Objeto perteneciente a la clase Perro.	Es compatible con su propia clase y con clases base según la herencia.	2
integer[]	Lista	Colección cuyos elementos son integer.	Se asigna a otra lista con elementos compatibles.	1

Tipos determinados en el programa

Figura 9: Sistema de tipos activo y tipos determinados en el programa.

Diagnósticos 0 Árbol sintáctico Tabla de símbolos Sistema de tipos Manejo de ámbitos Tokens Clases					
#	CATEGORÍA	TOKEN ANTLR	LEXEMA	LÍNEA	COLUMNA
1	Palabra reservada	'class'	class	1	1
2	Identificador	Identifier	Animal	1	7
3	Delimitador	'{'	{	1	14
4	Palabra reservada	'let'	let	2	3
5	Identificador	Identifier	nombre	2	7
6	Separador	':'	:	2	13
7	Tipo de dato	'string'	string	2	15
8	Fin de sentencia	';'	;	2	21
9	Palabra reservada	'function'	function	4	3
10	Identificador	Identifier	constructor	4	12
11	Delimitador	'('	(	4	23
12	Identificador	Identifier	nombre	4	24
13	Separador	':'	:	4	30
14	Tipo de dato	'string'	string	4	32
15	Delimitador	')'	)	4	38
16	Delimitador	'{'	{	4	40

Figura 10: Tokens producidos por el lexer de ANTLR, con categoría, lexema y posición.

Diagnósticos 0 Árbol sintáctico Tabla de símbolos Sistema de tipos Manejo de ámbitos Tokens Clases					
Animal Sin clase base Campos: nombre Métodos: constructor hablar			Perro Hereda de Animal Campos: — Métodos: hablar		

Figura 11: Clases declaradas con su clase base, campos y métodos.

10. Batería de pruebas

La batería está en `backend/tests/compiscript_tests.cpp` y contiene **95 escenarios**: 41 programas válidos y 54 errores esperados. No depende de ningún framework externo; enlaza directamente contra `compiscript_core`, de modo que prueba el mismo servicio que usan el CLI y el IDE.

El criterio de aprobación de un caso negativo es estricto: no basta con que el programa falle, sino que debe fallar **con el código de diagnóstico correcto**. Un programa que fuese rechazado por la razón equivocada se contaría como fallo.

Categoría	Aprobadas	Total
Proceso completo	4	4
Sistema de tipos	20	20
Manejo de ámbito	7	7
Funciones y procedimientos	13	13
Control de flujo	20	20
Clases y objetos	13	13
Listas y estructuras de datos	11	11
Reglas generales	7	7
Total	95	95

Tabla 7: Resultado de la batería por categoría.



Figura 12: Batería ejecutada desde el IDE: 95/95, con 41 casos válidos, 54 errores esperados y 0 fallos reales.

La batería también puede ejecutarse desde la línea de comandos:

```
1 ctest --test-dir build/compiscript --output-on-failure
```

Listing 5: Ejecución desde CTest.

11. Compilación y ejecución

11.1. Requisitos

- CMake 3.20 o superior.
- Compilador compatible con C++17.
- Java, únicamente para ejecutar la herramienta generadora de ANTLR.
- Python 3 con Flask, únicamente para servir el IDE.

11.2. Construcción

Un solo script descarga el JAR oficial de ANTLR si hace falta, regenera el lexer, el parser y el Visitor, descarga y compila el runtime C++ de ANTLR, construye `compiscript_cli` y `compiscript_tests`, y ejecuta la batería:

```
1 python3 -m pip install -r requirements.txt
2 ./scripts/build_compiscript.sh
```

Listing 6: macOS o Linux.

```
1 python -m pip install -r requirements.txt
2 powershell -File scripts/build_compiscript.ps1
```

Listing 7: Windows PowerShell.

11.3. Ejecución del IDE

```
1 python app.py
```

Listing 8: Servidor del IDE.

El IDE queda disponible en `http://127.0.0.1:5050`.

11.4. Uso directo del CLI

```
1 ./compiscript_cli --file backend/examples/compiscript/bienvenido.cps
2 ./compiscript_cli --stdin program.cps < program.cps
```

Listing 9: Análisis desde la línea de comandos.

La salida es un único documento JSON. El código de salida es 0 cuando no hay errores, 1 cuando el programa contiene diagnósticos y 2 cuando el uso del CLI o el archivo de entrada no son válidos.

12. Decisiones sobre el material oficial

El enunciado, la especificación y la gramática entregada no coinciden entre sí en tres puntos. Se resolvieron sin retirar construcciones oficiales.

- **Se agregó float.** La rúbrica exige operaciones entre `integer` y `float`, pero la gramática base solo contiene enteros. La gramática activa añade el tipo, el literal decimal y la ampliación implícita `integer` $\rightarrow$ `float`.
- **+ acepta dos string.** Los ejemplos oficiales usan concatenación. No se permite, en cambio, concatenar implícitamente cualquier objeto con un string.
- **Los cuerpos de control aceptan bloque o una sola sentencia.** El ejemplo oficial de recursión usa `if (n <= 1) return 1;` sin llaves, mientras la gramática base exigía bloque.

Existe además una contradicción explícita: la rúbrica exige que la condición de `switch` sea `boolean`, mientras que el documento de ejemplos usa un selector `integer`. **La implementación prioriza la regla explícita de la rúbrica.** La comprobación de compatibilidad de los casos está separada de la comprobación del selector, de modo que revertir esta decisión requiere retirar una sola llamada a `requireType`.

Por coherencia con la misma regla, `break` dentro de un `switch` que no está en un bucle también se reporta, porque el enunciado limita `break` y `continue` a bucles.

13. Correspondencia con la rúbrica

Componente	Pts	Evidencia en el proyecto
IDE	15	Editor con carga, guardado y descarga de <code>.cps</code> , análisis bajo demanda, diagnósticos navegables y siete paneles de resultados.
Analizador sintáctico y semántico	60	Gramática y código generado por ANTLR con <i>target</i> C++, árbol sintáctico visual, Visitor semántico de 1 218 líneas con 32 códigos de diagnóstico y batería de 95 escenarios.
Tabla de símbolos	25	Símbolos y scopes enlazados por identificador estable, resolución por cadena de ámbitos, registro de capturas de closures y vistas jerárquica y tabular por entorno.
Total	100	

Tabla 8: Evidencia por componente evaluado.

Respecto a los entregables solicitados: el repositorio conserva commits individuales por integrante (sección 4), la batería de pruebas cubre casos exitosos y fallidos de cada regla semántica (sección 10), la arquitectura y las instrucciones de ejecución están documentadas (secciones 3 y 11, además de `docs/arquitectura-compiscript.md` y `README.md`), y el IDE es funcional (sección 9).

14. Conclusiones

1. Mantener un único núcleo C++ consumido por el CLI, las pruebas y el servidor web eliminó la posibilidad de que la interfaz y la batería midieran comportamientos distintos. Es la decisión de diseño que más simplificó la depuración.
2. El patrón Visitor resultó adecuado para el análisis semántico porque permite sintetizar el tipo de una expresión desde sus subexpresiones, devolviéndolo hacia el nodo padre. Con un Listener habría sido necesario administrar manualmente una pila de resultados.
3. Conservar los scopes después de salir de cada bloque, en lugar de destruirlos, convirtió la tabla de símbolos en una instantánea completa y navegable del programa. Esto no solo satisface la evaluación: es la estructura que las fases posteriores del compilador necesitarán.
4. Suspender el análisis semántico cuando hay errores sintácticos evita diagnósticos en cascada sobre un árbol incompleto, sin perder la información útil, porque los tokens y el árbol se siguen entregando.
5. Las contradicciones entre el enunciado, la especificación y la gramática entregada obligaron a tomar decisiones explícitas. Documentarlas y aislarlas en el código —el caso de `switch` se revierte retirando una sola llamada— resultó más valioso que elegir en silencio.

15. Referencias

- Aho, A., Lam, M., Sethi, R. y Ullman, J. *Compilers: Principles, Techniques, and Tools*. 2.^a edición, 2006. Secciones 2.7 (tablas de símbolos), 6.5 (comprobación de tipos) y 7.3 (acceso a datos no locales).

- Parr, T. *The Definitive ANTLR 4 Reference*. Documentación del *target* de C++: <https://github.com/antlr/antlr4>.
- Enunciado del proyecto: *Análisis Semántico de Compiscript*, Construcción de Compiladores, agosto de 2026.
- Documentación interna del repositorio: `docs/arquitectura-compiscript.md` y `docs/decisiones-compiscript.md`.